Day 4: Real-World QA Workflow

Agile & Scrum ceremonies, User Stories, Acceptance Criteria, Jira boards, exploratory testing, and test summary reports.

1. Day Learning Objectives

- [ ] Navigate the QA role within Agile & Scrum workflows (Sprints, Stand-ups, Refinements, Retrospectives)
- [ ] Analyze User Stories and Acceptance Criteria to uncover hidden ambiguities and gaps
- [ ] Execute structured Exploratory Testing using timeboxed test charters
- [ ] Apply Risk-Based Testing to prioritize test execution under tight release deadlines
- [ ] Formulate objective Entry and Exit criteria for testing phases
- [ ] Draft an executive Test Summary Report with data-backed Go / No-Go recommendations

2. Core Concepts & Practical Fundamentals

1. Agile & Scrum Ceremonies for QA Engineers

In Agile development, teams deliver working software in short iterations (Sprints, usually 2 weeks). QA is embedded in the team:

• Sprint Planning: QA estimates test effort and reviews acceptance criteria.
• Daily Stand-up: QA answers: 1. What did I test yesterday? 2. What will I test today? 3. Are there any blockers?
• Sprint Review / Demo: Working software is demonstrated to stakeholders.
• Sprint Retrospective: Team discusses what went well, what failed, and action items for improvement.

Real-World Workplace Examples:

• **[Stand-up Communication]:** "Yesterday I completed test case execution for user profile updates. Today I am testing password reset flows and verifying bug #104. I am currently blocked on staging API database connectivity."

Key Standard Terminology:

• **Sprint:** A timeboxed iteration (typically 1–4 weeks) during which a specific set of work is completed.
• **Blocker:** An obstacle that halts test execution or development progress.
• **Definition of Done (DoD):** Agreed-upon criteria that must be satisfied before a user story is marked complete (including QA sign-off).

Common Beginner Mistakes & Interview Traps:

& Waiting until day 8 of a 10-day sprint to begin testing.
& Not speaking up during sprint planning when acceptance criteria are ambiguous.

2. User Stories & Acceptance Criteria Analysis

A User Story follows the template:
"As a [user persona], I want [capability] so that [benefit]."

Acceptance Criteria (AC) define boundaries of acceptance (often written in Given-When-Then Gherkin format or clear bullet points). QA's primary duty is finding ambiguities in AC before development begins:

• What happens on network timeout?
• Can a coupon be used multiple times?
• What is the maximum character limit?

Real-World Workplace Examples:

• **[Gift Card Checkout Story]:** Story: "As a shopper, I want to apply a gift card during checkout so my total payment decreases."
Ambiguities QA catches: What if gift card balance exceeds order total? Is remainder saved or forfeited? Can multiple gift cards be combined?

Key Standard Terminology:

• **User Story:** A short, simple description of a feature told from the perspective of the user.
• **Acceptance Criteria:** The explicit conditions that software must fulfill to be accepted by the product owner.

Common Beginner Mistakes & Interview Traps:

& Assuming developer knows unstated requirements without clarifying them.
& Treating acceptance criteria as a rigid ceiling rather than a baseline for testing.

3. Exploratory Testing & Risk-Based Testing

• Exploratory Testing: Simultaneous learning, test design, and test execution without pre-written scripted test cases. Governed by a timeboxed Test Charter (e.g., 45 minutes targeting checkout error handling).

• Risk-Based Testing: Prioritizing test cases based on Risk = Probability of Failure × Impact of Failure. High-risk modules (Payments, Auth, Data retention) receive 80% of test effort, while low-risk features (Footer links) receive basic sanity checks.

Real-World Workplace Examples:
• **[Test Charter Example]:** Charter: Explore the shopping cart coupon input with special characters and rapid clicking to discover UI freezing or duplicate discount bugs. Timebox: 30 minutes.

Key Standard Terminology:
• **Exploratory Testing:** An approach to testing where testers dynamically design and execute tests based on system discovery.
• **Risk-Based Testing:** Organizing and prioritizing test execution based on calculated failure risk.

Common Beginner Mistakes & Interview Traps:
& Confusing exploratory testing with random chaotic "ad-hoc monkey testing". Exploratory testing requires a defined charter and documented findings.

4. Entry/Exit Criteria & The Test Summary Report

• Entry Criteria: Prerequisites before testing can start (e.g. build deployed to staging, unit tests pass at 85%, test data seeded).
• Exit Criteria: Conditions before testing can stop (e.g. 100% test cases executed, 0 Blocker/Critical defects open, 95% pass rate).

• Test Summary Report: Formal executive document delivered before release:
1. Scope of testing
2. Metrics: Total executed, Passed, Failed, Blocked
3. Outstanding open defects & risk assessment
4. Formal Recommendation: Go, No-Go, or Go with Conditions

Real-World Workplace Examples:
• **[Release Sign-Off Meeting]:** QA Lead presents: 45 test cases executed, 43 passed, 2 minor cosmetic defects open (workaround available). 0 Blocker/Critical bugs. Recommendation: GO for v2.4 production deployment.

Key Standard Terminology:
• **Exit Criteria:** The set of conditions that permit a testing phase to be declared complete.
• **Test Summary Report:** An executive artifact summarizing testing activities, results, and release recommendations.

Common Beginner Mistakes & Interview Traps:
& Recommending "Go" when critical payment bugs remain open without management sign-off.

3. Guided Practice Exercise

Guided Exercise: Reviewing a Simulated User Story for Gaps

Analyze a user story for a loyalty points redemption feature and document 4 critical edge questions.

Feature Specification Under Test:

USER STORY:
As a frequent shopper, I want to redeem my store points at checkout to receive a cash discount.

ACCEPTANCE CRITERIA:
1. 100 points = $1.00 discount.
2. User selects number of points to redeem.
3. Discount applies to cart total.

Recommended Execution Steps:
• Step 1: Check minimum/maximum constraints: Can user redeem 1 point ($0.01)? Is there a maximum redemption cap?
• Step 2: Check balance boundary: What happens if user inputs more points than they possess?
• Step 3: Check order cancellation: If an order paid with points is cancelled or refunded, are points restored immediately or forfeited?
• Step 4: Check concurrent sessions: What happens if user logs in on 2 devices simultaneously and redeems the same points twice?

4. Practical Workplace Assignment & Evaluation Rubric

Day 4 Assignment: User Story Analysis, Scenario Design & Test Summary

Analyze a simulated Agile user story, identify at least 2 ambiguities, document 4 test scenarios, record execution results, and write an executive Test Summary Report with a release recommendation.

System Under Test Brief:
User Story: Gift Card Redemption at Checkout. Acceptance criteria states customer can apply gift card code to reduce balance. System has 2 subtle gaps: behavior when gift card balance exceeds order total, and behavior on cancelled orders.

Submission Instructions:
• Identify and write down at least 2 clear requirement ambiguities or clarifying questions for the Product Owner.
• Design at least 4 test scenarios covering full balance, partial balance, expired gift card, and cancelled order refund.
• Document test results and author a concise Test Summary Report including: Total Executed, Pass Rate, Open Defects, and an executive Go / No-Go release recommendation.

Deterministic Evaluation Rubric:
' **[Weight 30%] Ambiguities Identified [Requires at least 2]:** Learner identifies at least 2 genuine requirement gaps or open questions.
' **[Weight 35%] Test Scenarios Formulated [Requires at least 4]:** Formulates at least 4 practical test scenarios covering edge states.
' **[Weight 35%] Executive Test Summary Report:** Includes test metrics, open defect summary, and clear Go/No-Go release recommendation.

5. Industry Interview Questions & Model Answers

Q1 [Workplace Behavioral]: What do you do if you find a critical defect two hours before a scheduled production release?

Model Answer:

1. Immediately verify the bug on a clean environment to ensure reproducibility and gather evidence (logs, screenshots).
2. Alert the QA Lead, Engineering Lead, and Product Owner immediately without waiting for stand-up.
3. Log an urgent Blocker defect ticket in Jira with complete steps.
4. Participate in defect triage: assess whether a hotfix is feasible or if the release must be aborted (No-Go decision).
5. Never silently ignore or sign off on a known critical defect without formal executive approval.

Avoid this mistake: Panicking, trying to fix the code yourself, or hiding the defect to meet the deadline.

Q2 [Scenario]: How do you handle a situation where a developer rejects your bug as "Working as Designed"?

Model Answer:

1. Do not argue confrontationally.
2. Review the User Story and Acceptance Criteria to verify if the specification supports your expected result.
3. Re-test on a clean browser or staging environment to ensure environment parity.
4. Pair with the developer: demonstrate the exact reproduction steps on screenshare.
5. If the requirement is ambiguous or silent, involve the Product Owner to provide final business clarification.

Avoid this mistake: Escalating immediately to executive management without discussing with the developer first.

6. Key Takeaways & Fast Revision Notes

Core Takeaways:

- QA actively participates in sprint ceremonies to prevent defects early.
- Acceptance Criteria must be challenged for edge cases before coding begins.
- Exploratory testing is structured learning guided by timeboxed charters.
- The Test Summary Report provides stakeholders with objective data to make Go / No-Go decisions.

Cheat Sheet / Revision Notes:

- Scrum Ceremonies: Sprint Planning, Daily Stand-up, Sprint Review, Sprint Retrospective.
- Risk-Based Testing = Likelihood of Failure × Impact of Failure.
- Test Summary Report contents: Scope, Metrics, Open Defects, Residual Risks, Recommendation.